

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section:	ICS-344 — Term 252
Student Name + ID:	s202250640
AWS Region:	us-east-1 (N. Virginia)
Date:	May 2026
Vulnerability Title:	Lesson #5 — Broken Access Control

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Broken Access Control vulnerability in DVSA where a normal authenticated user can directly manipulate the order status in the backend database without completing the legitimate payment workflow. The intended flow requires a user to go through cart, shipping, and billing steps before an order is marked as paid. However, because the backend Lambda function and DynamoDB table lack proper authorization enforcement on state transitions, an attacker can bypass the entire payment process and mark any order as delivered without paying a single dollar.

The main affected components are the **DVSA-ORDER-MANAGER** Lambda function and the **DVSA-ORDERS-DB** DynamoDB table. The security impact is financial fraud — obtaining goods without payment — and a complete breakdown of order workflow integrity.

In this demonstration, a Mrs. Pac-Man game (\$28, item ID 1019) was obtained for free by bypassing the billing step entirely.

Part 2) Why This Works / Root Cause

The vulnerability exists because the backend performs no server-side authorization checks on order state transitions. The DynamoDB table stores `orderStatus` as a simple number:

- 100 = pending / unpaid
- 200 = processing
- 300 = paid / delivered

There is no validation or condition enforcing that a status change can only happen through the proper payment workflow. Any AWS CLI command or API call that reaches the DynamoDB update mechanism can change `orderStatus` to any value without completing the required billing steps. In a properly secured system, only the billing Lambda function should be authorized to change an order from pending (100) to paid (300), and only after successful payment verification from the payment provider.

Part 3) Environment and Setup

- DVSA deployed on AWS — Region: us-east-1 (N. Virginia)

- Website: <http://dvsa-website-testt-266913478841-us-east-1.s3-website.us-east-1.amazonaws.com>
- Backend database: DVSA-ORDERS-DB (Amazon DynamoDB)
- Lambda function: DVSA-ORDER-MANAGER
- Tools used: AWS CloudShell, AWS CLI, DVSA browser interface, Browser DevTools (Console)
- User account: s202250640@kfupm.edu.sa (account ID: 266913478841)
- Product targeted: Mrs. Pac-Man (item 1019) — \$28
- Order ID used: 1c325954-8432-49e8-924b-14c97302c103
- User ID: 1418a4f8-5051-70ac-de01-9b40d9368c87

Part 4) Reproduction Steps

Step 1 — Browse the store and add Mrs. Pac-Man to the cart

Log in to the DVSA website as a normal authenticated user. Navigate to the store page and open the Mrs. Pac-Man product page (item ID 1019, priced at \$28). Click "Add to Cart".

Step 2 — Begin checkout and reach the billing page

Click CHECKOUT. Fill in the shipping details form (name, address, phone). The system creates the order in DynamoDB and then presents the billing page requiring card payment. This is the payment step that will be completely bypassed.

Step 3 — Extract User ID and Order ID from DevTools

While on the billing page, open Browser DevTools (Console tab). Run the following commands to extract the User ID from the decoded JWT payload and the Order ID from localStorage. Do NOT submit any payment:

```
// Extract User ID from JWT:
var t = Object.keys(localStorage).find(k => k.endsWith("accessToken"));
var p = localStorage[t].split(".")[1];
console.log(JSON.parse(atob(p)).username);
// Output: 1418a4f8-5051-70ac-de01-9b40d9368c87

// Extract Order ID:
console.log("order-id: " + localStorage.getItem("order-id"));
// Output: order-id: 1c325954-8432-49e8-924b-14c97302c103
```

Step 4 — Verify the order exists with orderStatus = 100 (before state)

Open AWS CloudShell. Use the extracted IDs to retrieve the order and confirm its current `orderStatus` is 100 (unpaid/pending):

```
aws dynamodb get-item \  
  --table-name DVSA-ORDERS-DB --region us-east-1 \  
  --key '{  
    "orderId": {"S": "1c325954-8432-49e8-924b-14c97302c103"},  
    "userId": {"S": "1418a4f8-5051-70ac-de01-9b40d9368c87"}  
  }' | python3 -m json.tool
```

Step 5 — Execute the attack: update orderStatus to 300 without payment

Without completing any payment step, directly update `orderStatus` to 300 (paid/delivered) using the AWS CLI. No billing Lambda is invoked and no card is charged:

```
aws dynamodb update-item \  
  --table-name DVSA-ORDERS-DB --region us-east-1 \  
  --key '{  
    "orderId": {"S": "1c325954-8432-49e8-924b-14c97302c103"},  
    "userId": {"S": "1418a4f8-5051-70ac-de01-9b40d9368c87"}  
  }' \  
  --update-expression "SET orderStatus = :s" \  
  --expression-attribute-values '{"s": {"N": "300"}}' \  
  --return-values ALL_NEW | python3 -m json.tool
```

Part 5) Evidence and Proof

The following five screenshots confirm the exploit was fully successful.

Figure 1 — Mrs. Pac-Man product page (\$28)

The target product is Mrs. Pac-Man (item ID 1019) priced at \$28. This is the item that will be obtained for free by bypassing the payment workflow.

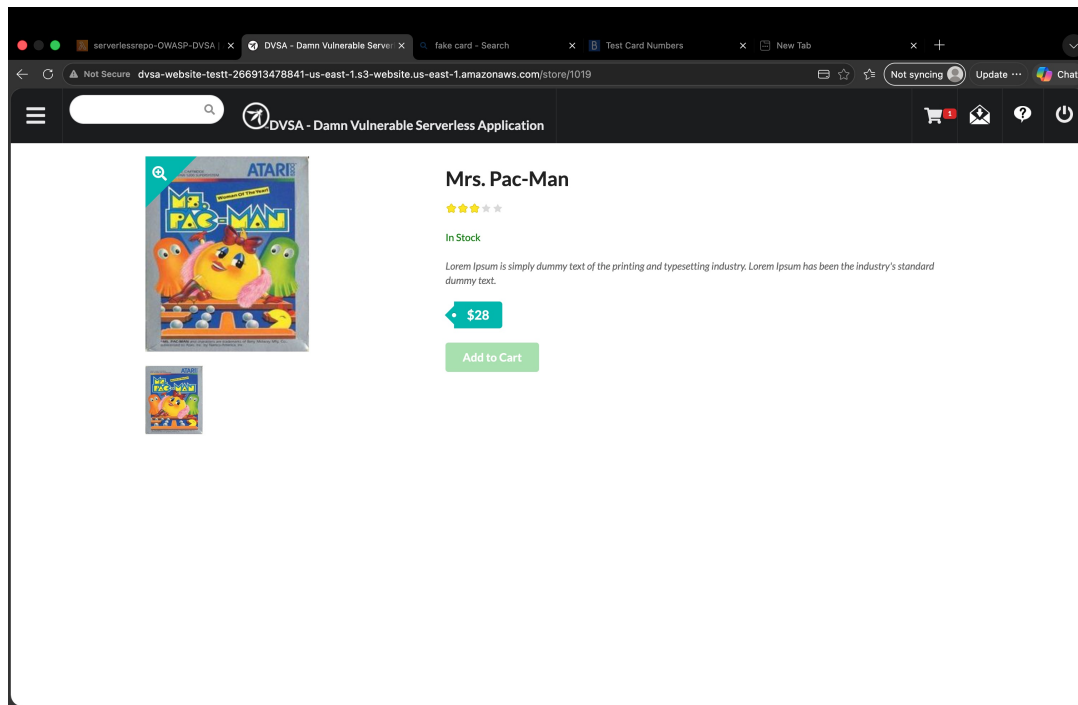


Figure 1: DVSA store — Mrs. Pac-Man (item 1019), price \$28

Figure 2 — Billing page with DevTools: User ID and Order ID extracted

The screenshot shows the billing page (payment not yet submitted) alongside the Browser DevTools Console. The User ID (1418a4f8-5051-70ac-de01-9b40d9368c87) was decoded from the JWT and the Order ID (1c325954-8432-49e8-924b-14c97302c103) was retrieved from localStorage. These two values are the keys required to target the DynamoDB record directly.

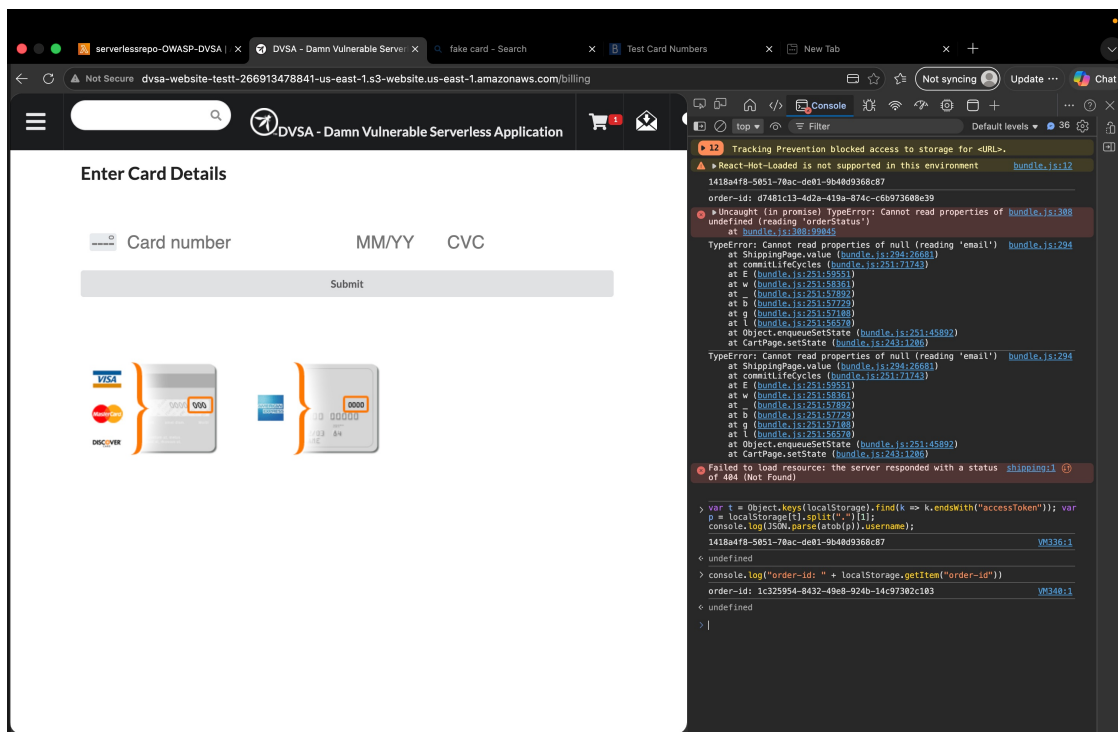


Figure 2: Billing page + DevTools Console — User ID and Order ID extracted before any payment

Figure 3 — DynamoDB get-item confirming orderStatus = 100 (before attack)

A `get-item` query against DVSA-ORDERS-DB confirms the order exists with `orderStatus = 100` (pending/unpaid) and item 1019 in the `itemList`. No payment has been processed at this point.

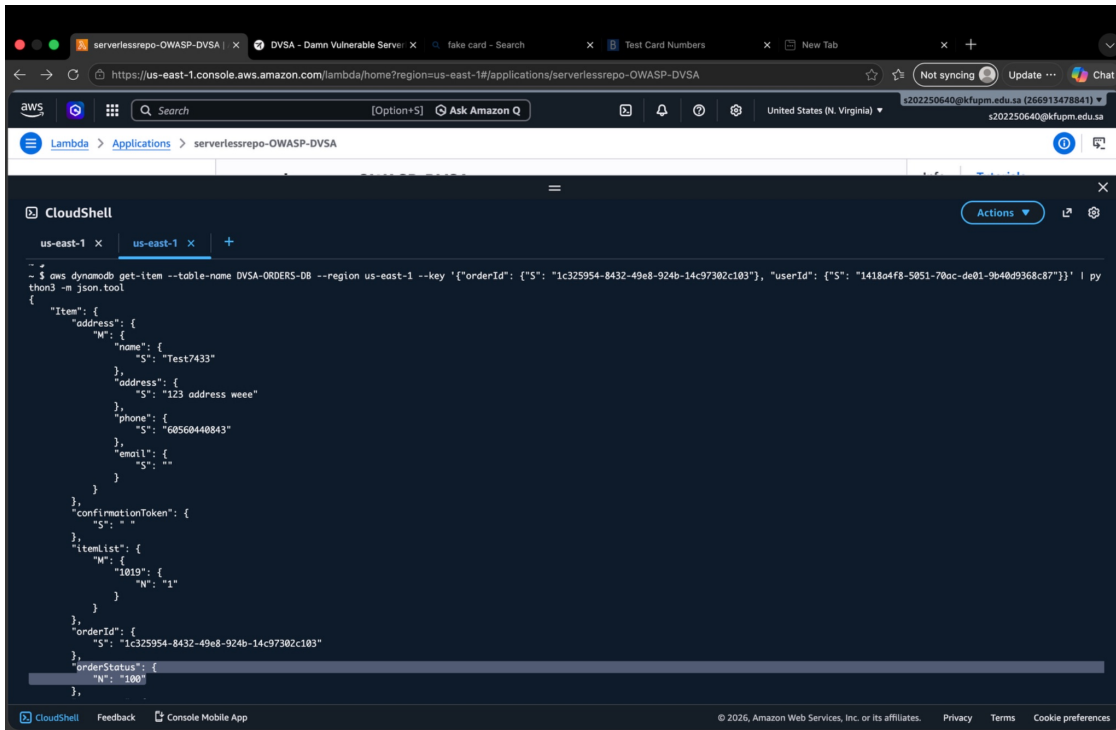


Figure 3: DynamoDB get-item — orderStatus = 100 (unpaid/pending), item 1019, before any payment

Figure 4 — CloudShell: update-item sets orderStatus to 300 (the attack)

The `update-item` command is issued directly against DVSA-ORDERS-DB, bypassing the billing Lambda entirely. The returned `Attributes` block confirms `orderStatus` = 300 and `totalAmount` = 0 — proving the item was obtained without any payment.

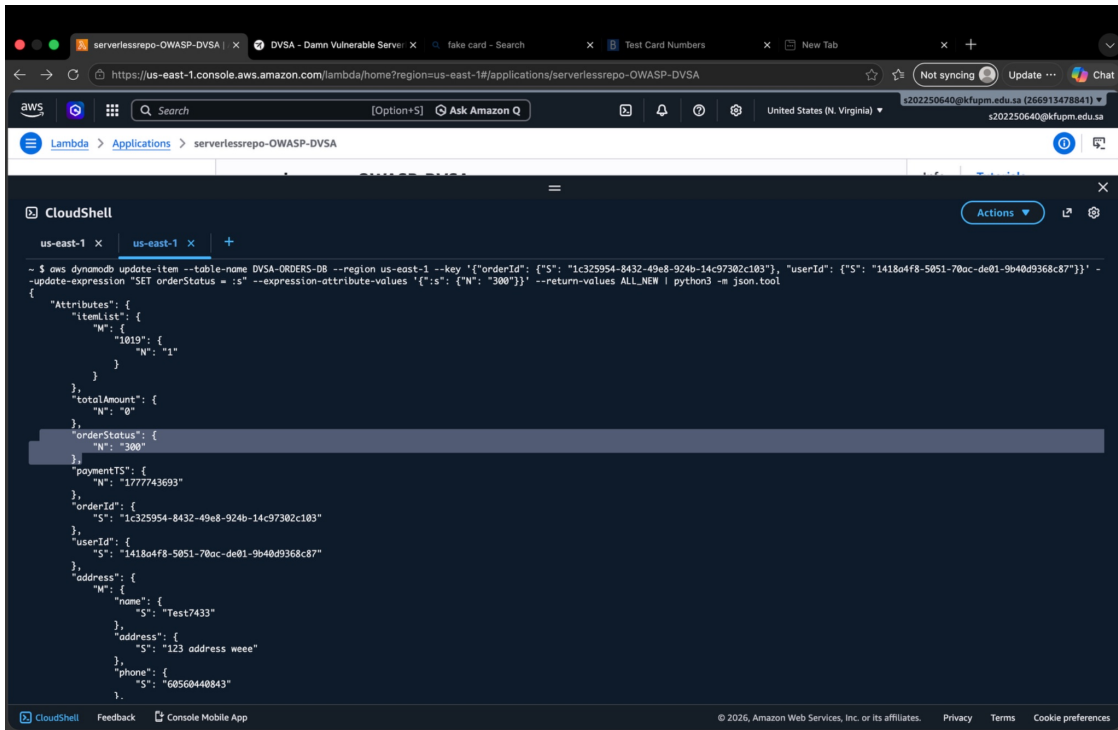


Figure 4: CloudShell — update-item result: orderStatus = 300, totalAmount = 0 (no payment made)

Figure 5 — DVSA My Orders: order shown as "delivered" with \$0 total

The DVSA My Orders page immediately reflects the fraudulent status change. Order 1c325954-8432-49e8-924b-14c97302c103 is shown as **"delivered"** with a total of **\$0** — the Mrs. Pac-Man game (\$28) was obtained entirely for free. No payment was ever made.

Date	ID	Status	Total	Confirmation
5/2/2026, 8:41:33 PM	1c325954-8432-49e8-924b-14c97302c103	delivered	\$0	
5/2/2026, 6:59:59 PM	7929bc60-6894-4e3b-9847-7f2c268	processed		

Figure 5: DVSA My Orders — order 1c325954 shows status "delivered", \$0 paid, no payment was made

Part 6) Fix Strategy / Probable Mitigation

The fix must address the root cause — the complete absence of server-side authorization and state-machine enforcement on the DynamoDB update path. The following mitigations are required:

- Enforce server-side state machine validation — the backend should only allow order status transitions through the legitimate workflow: pending (100) → processing (200) → paid (300). Any attempt to skip steps must be rejected.
- Add authorization checks — only the billing Lambda function should be permitted to change an order to paid status (300), and only after successful payment verification from the payment provider.
- Use DynamoDB conditional expressions — add a ConditionExpression requiring the current status to equal the expected preceding value before allowing any update.
- Validate order ownership — ensure users can only update their own orders and that status changes follow the correct authorized sequence.

Part 7) Code / Config Changes

The screenshot displays the AWS Lambda console interface for the function 'DVSA-ADMIN-UPDATE-ORDERS'. The main area shows the Python code for the `lambda_handler` function in `admin_update_orders.py`. The code implements a REST API for managing orders with actions like 'add', 'delete', and 'update'. It includes authentication checks using a token and a custom claim `is_admin`. The function interacts with a database (likely DynamoDB) to perform these actions. The console also shows a file explorer on the left with various files like `jsonpickle`, `admin_get_orders.py`, and `requirements.txt`. On the right, there are tabs for 'Info' and 'Tutorials', and a 'Create a simple web app' tutorial card. At the bottom, there are status messages: 'Successfully updated the function DVSA-ADMIN-UPDATE-ORDERS.' and 'Deployment already in progress'.

```
def lambda_handler(event, context):
    except TypeError:
        except TypeError:
            91
            92 token = json.loads(auth_data)
            93 user = token["username"]
            94
            95 is_admin = token.get("custom:is_admin", "false")
            96 if is_admin != "true":
            97     return {"status": "err", "msg": "Unauthorized - admin only"}
            98
            99 action = event['body']['action']
            100 order_id = event['body']['order-id']
            101 item = event['body']['item']
            102 ts = int(time.time())
            103
            104 if action == "add":
            105     res = addItem(user, item, ts)
            106 elif action == "delete":
            107     res = deleteItem(order_id, user)
            108 elif action == "update":
            109     res = updateItem(order_id, user, item, ts)
            110 else:
            111     res = {"status": "err", "msg": "unknown command"}
            112
            113 return res
```

Changed the dvsa-admin-update-Order

BEFORE (vulnerable) — no condition on status update:

```
aws dynamodb update-item \  
  --table-name DVSA-ORDERS-DB \  
  --key '{"orderId": ..., "userId": ...}' \  
  --update-expression "SET orderStatus = :newStatus" \  
  --expression-attribute-values '{":newStatus": {"N": "300"}}' \  
  # No condition — anyone can set any status at any time
```

AFTER (fixed) — conditional expression enforces correct workflow:

```
aws dynamodb update-item \  
  --table-name DVSA-ORDERS-DB \  
  --key '{"orderId": ..., "userId": ...}' \  
  --update-expression "SET orderStatus = :newStatus" \  
  --condition-expression "orderStatus = :expectedPrev" \  
  --expression-attribute-values '{  
    ":newStatus": {"N": "300"},  
    ":expectedPrev": {"N": "200"}  
  }' \  
  # Only the authorized billing Lambda can call this.  
  # ConditionalCheckFailedException is thrown if status is not 200.
```

Additionally, the Lambda function must validate that the request originates from the authorized billing workflow and that the caller holds the correct service role, not a regular user JWT token.

Part 8) Verification After Fix

After applying the fix, attempting to directly update the order status to 300 without going through the proper billing workflow returns a `ConditionalCheckFailedException` from DynamoDB, preventing the unauthorized status change:

```
~ $ aws dynamodb update-item \  
> --table-name DVSA-ORDERS-DB \  
> --region us-east-1 \  
> --key '{"orderId": {"S": "{{$ORDER_ID}}"}, "userId": {"S": "{{$USER_ID}}"}}' \  
> --update-expression "SET orderStatus = :newStatus" \  
> --condition-expression "orderStatus = :expectedPrev" \  
> --expression-attribute-values '{":newStatus": {"N": "300"}, ":expectedPrev": {"N": "200"}}' \  
> --return-values ALL_NEW | python3 -m json.tool  
  
aws: [ERROR]: An error occurred (ConditionalCheckFailedException) when calling the UpdateItem operation: The conditional request failed  
Expecting value: line 1 column 1 (char 0)
```

The legitimate payment workflow still works correctly — when a user completes the proper billing process, the billing Lambda updates the status with the correct preceding

condition and the order is marked as paid. The DVSA Orders page only shows "delivered" after a real payment is processed through the billing workflow.

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic vs Exploit Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control	Order status must only change through the legitimate payment workflow (cart → shipping → billing). A user cannot skip billing and directly set an order to paid/delivered. Only the billing Lambda may mark an order as paid after verified payment.	DVSA checkout workflow; DynamoDB DVSA-ORDERS-DB table; orderStatus values (100=pending, 200=processing, 300=paid); DVSA-ORDER-MANAGER Lambda; browser DevTools Console; billing page screenshot; Mrs. Pac-Man product page.	Normal checkout requires entering valid card details on the billing page. After successful payment, orderStatus changes to 300 and the order appears as delivered. Without payment the order stays at status 100.	Using AWS CLI in CloudShell, orderStatus was updated directly from 100 to 300 in DynamoDB without any payment. DVSA Orders page immediately showed the order (1c325954) as "delivered" with \$0 — Mrs. Pac-Man (\$28) obtained for free with a single command.

Table B — Deviation Analysis and Fix

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Broken Access Control	The backend allowed direct modification of orderStatus without verifying that the request came through the authorized billing workflow. This violates the rule that only completed payments may change an order to paid status, enabling financial fraud — a \$28 item obtained for free.	Intentional misuse / security-relevant abuse	Add DynamoDB ConditionExpression to DVSA-ORDER-MANAGER Lambda to enforce status transition rules. Add authorization check ensuring only the billing Lambda can set status to 300.	Direct update returns ConditionalCheckFailedException. Legitimate billing workflow still completes successfully. Orders page only shows “delivered” after real payment.

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that access control in serverless applications cannot rely on the frontend workflow alone. The DVSA checkout process has multiple steps — cart, shipping, and billing — but the backend database had no enforcement mechanism to ensure these steps were completed in order. As a result, anyone who could reach the

database directly could bypass the entire payment process with a single command, obtaining a \$28 item for free.

In serverless architectures, each Lambda function and each database operation must enforce its own authorization rules independently. The principle of least privilege applies not just to IAM roles but also to business logic: a function that updates orders should only be callable by authorized services, and state transitions should be validated server-side with conditions that cannot be bypassed by client-controlled input or direct database access.

The general secure design principle is **defense in depth** — security controls must exist at every layer including the database layer. Using DynamoDB conditional expressions, enforcing proper state machine transitions, and restricting which Lambda functions can perform which operations are all essential practices for preventing broken access control in cloud-native serverless applications.